

Artificial Intelligence 403-1

Project Title: Pac-Man Pathfinding

Student Name: Aida Jafari

Student ID: 40007653

1. Introduction

This project involves creating a simple Pac-Man pathfinding simulation as part of my Artificial Intelligence 403-1 course. The agent (Pac-Man) moves through a grid filled with walls, open spaces, and food. The objective is for Pac-Man to reach the food using Depth-First Search (DFS) while keeping track of each location it has visited along the way.

2. Project Overview

The project begins by reading the maze setup from a file. The grid layout includes walls ('*'), open paths ('-'), the agent ('a'), and the food ('f'). The program uses DFS to explore the maze and records its journey to the food. Additionally, it generates a “memory map” of visited cells, which shows only what Pac-Man has explored.

3. Input and Output Explanation

The program reads an input file that includes:

Maze Dimensions: The first line specifies the grid's rows and columns (e.g., 6,10).

Agent's Start Position: The second line gives the agent's starting location as a coordinate, like (3,2).

Maze Layout: The rest of the file shows the maze using symbols ('*', '-', 'a', and 'f'), where each symbol represents a wall, an open path, the agent, or food.

After running DFS, the program outputs:

Start Position: (3,2) (where the agent begins).

Food Position: (3,6) (location of the food).

Number of Moves: Total moves taken to find the food.

Move Sequence: Step-by-step path taken by the agent.

Memory Map: A visual representation of the agent's explored paths, with unvisited cells marked as '?'.

For example, given the path (3,2) -right-> (3,3) -up-> (2,3), the memory map will show all cells that Pac-Man visited while keeping unknown areas as '?'.

4. Core Functions Explained

`is_valid_position(maze, position)`

This function checks if a move is within the grid and not hitting a wall. It makes sure Pac-Man only moves to open spaces.

`read_map_from_file(file_name)`

This function reads the maze from a .txt file, setting up the maze, the agent's start location, and the food position based on the symbols provided in the input file.

`dfs(maze, current_position, goal_position, path, visited, memory)`

This is the main DFS function, responsible for moving the agent step-by-step until it finds the food. It updates the path and tracks visited cells in the memory map, backtracking if it hits dead ends.

`main()`

Handles reading the input file, setting up the DFS search, and printing the final results, including the memory map after the agent reaches the food.

5. DFS Search Process

Using DFS, the agent explores each possible path in the maze. DFS is suitable here because it allows the agent to go deep into each path, backtracking if it encounters walls or dead ends. This means Pac-Man will not necessarily take the shortest route but will eventually reach the food through an exhaustive search of possible paths.

6. Memory Map Output

The memory map is an essential part of the project. After the agent finds the food, it outputs a revised maze showing where Pac-Man has been. Cells it visited are shown as '-', walls are still '*', the food is 'f', and unknown cells are marked with '?'. This provides a simple, human-readable way to see Pac-Man's journey through the maze.

7. Conclusion

This project showcases a basic implementation of DFS for pathfinding in a grid maze. By simulating Pac-Man's journey to food, the code demonstrates fundamental AI pathfinding

techniques and how to track exploration with a memory map. Future enhancements could involve using different search strategies like Breadth-First Search (BFS) or A* to find shorter paths more efficiently.